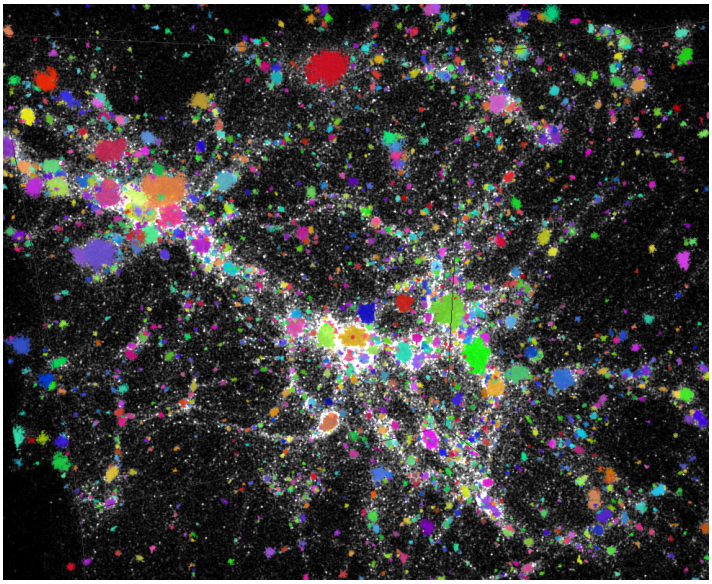
CosmoViewer

An interactive workbench for particle simulations of the universe: explore a simulation in 3D, pick out the structures that matter, follow them through time and measure them, in one window. Python · GNU GPL v3 · Gadget, EAGLE, AGORA and SWIFT snapshots.



DM-L50-N128 at z = 0, rendered by CosmoViewer's movie maker; particles coloured by Gadget-4 FoF group.

The problem

Viewers show particles; scripts measure them. The hand-off between the two is where errors creep in, and the worst produce a convincing picture with a wrong number. A halo selected from a drawn sample keeps its size and dispersion but loses mass in proportion to the sample; catalogue group numbers change between snapshots; a forgotten *h* shifts every size by about 1.5.

The model underneath

CosmoViewer keeps three things apart: the **population** (particles in the simulation), the **representation** (what is drawn) and the **selection** (what is measured).

> *What you see may be sampled, smoothed or interpolated, but what you measure never is.*

A selection records how it was made, on which snapshot, how many members it should have and whether it has them all. It is found again by ParticleID in any snapshot, and every analysis of an incomplete selection says so.

What it does

- **See.** Level-of-detail rendering with a fixed point budget spent where you look; snapshots larger than memory stream from disk (850 million EAGLE particles on a laptop). A 2D slice or smoothed map follows the 3D camera.
- **Find.** Friends-of-Friends with Gadget-4's conventions in its own process; Gadget-4 catalogues read directly; a halo table linked to the view.
- **Follow.** Six selection sets tracked by ParticleID, through gas-to-star conversion; a particle inspector across every snapshot; trails unwrapped across the periodic box.
- **Measure.** 24 quantity plugins, radial profiles, Lagrangian radii with errors, energies and virial ratio, phase diagrams and a sanity report, all on the full population; results export as CSV or a script that reproduces them.
- **Show.** Quick and keyframed movie makers with burnt-in time, scale and colour bars.

Validation

Test	Result
FoF membership vs Gadget-4, DM-L50-N128 (3 snapshots)	8,030 of 8,032 groups identical; 2 float32 ties
Group mass and centre vs catalogues	Agree to float32 rounding
Energies vs Gadget-4 energy.txt	K, U and mass per type agree to all 8 digits
Virial ratio, Plummer spheres ($N = 10^5$)	0.4996 ± 0.0002 (0.5 expected)
Lagrangian radii, 100 Plummer realisations	93–98 % within 2σ (95.4 % expected)
Sanity report, 15 planted defects	All 15 caught
Phase diagram vs independent h5py code	Every bin equal to 10^{-12}

Performance

EAGLE Ref-L0025N0752 (850M particles, 16 files, 58 GiB): 1.5–2.0M of a 2M budget drawn from the whole box down to full resolution at the focus; 3.5–4.0 GB peak memory on a 16 GB MacBook. FoF on 2.1M particles: about 10–11 s, about 0.5 GB.

Built on

PyQt6, pyqtgraph and PyOpenGL for the interface and drawing; NumPy, SciPy and h5py for data; Astropy for units. Test suites in both applications, with real-data checks against Gadget-4 output.

Interaction model after GadgetViewer (J. Helly). FoF: Davis et al. 1985 with Gadget-4 conventions (Springel et al. 2021). Data: Gadget-4 examples; EAGLE public release (Schaye et al. 2015; McAlpine et al. 2016). Conceived, designed and directed by Arpit Pratap Singh; see the README for the AI disclosure.